Do you understand swagger openapi files

Yes, I'm familiar with Swagger/OpenAPI files. Swagger/OpenAPI is a specification for defining and documenting APIs. It allows you to describe the structure and behavior of your API endpoints, including request/response formats, authentication requirements, error handling, and more. Swagger/OpenAPI files are typically written in YAML or JSON format and serve as a contract between API providers and consumers.

With Swagger/OpenAPI files, you can generate interactive API documentation, generate client SDKs for different programming languages, and even perform automated testing and validation of your API. The specification provides a standardized way to describe APIs, making it easier for developers to understand and interact with different services.

If you have any specific questions about Swagger/OpenAPI files or need assistance with a particular aspect of them, feel free to ask!

Help me add tags to this open API file:

```
openapi: 3.0.3
info:
  title: Gloss Translation API
  description: |-
    This is the Gloss Translation API.
    The frontend can be accessed at [https:/
  version: 0.0.1
servers:
  - url: /api
paths:
  /auth/signin/{provider}:
    $ref: ./paths/auth.yml#login
  /auth/signout:
    $ref: ./paths/auth.yml#logout
  /auth/csrf:
```

```
    $ref: ./paths/auth.yml#csrf
/auth/session:
    $ref: ./paths/auth.yml#session
/users:
    $ref: ./paths/users.yml
/users/{userId}:
    $ref: ./paths/user.yml
/verses/{verseId}:
    $ref: ./paths/verse.yml
/languages:
    $ref: ./paths/languages.yml
/languages/{code}:
    $ref: ./paths/language.yml
/languages/{code}/verses/{verseId}/transl
    $ref: ./paths/translations.yml
/languages/{code}/verses/{verseId}/glosse
    $ref: ./paths/verse-glosses.yml
/languages/{code}/words/{wordId}:
    $ref: ./paths/language-words.yml
```

To add tags to the OpenAPI file, you need to specify the `tags` field within each endpoint definition. The `tags` field allows you to categorize and group related endpoints together. Here's an updated version of your OpenAPI file with tags added to each endpoint:

yaml Copy code

```yaml
openapi: 3.0.3
info:
  title: Gloss Translation API
  description: |-
    This is the Gloss Translation API.
    The frontend can be accessed at [https://gloss-translation.vercel.app]
  version: 0.0.1
servers:
  - url: /api
paths:
  /auth/signin/{provider}:
    $ref: ./paths/auth.yml#login
```

```
    tags:
      - Authentication
  /auth/signout:
    $ref: ./paths/auth.yml#logout
    tags:
      - Authentication
  /auth/csrf:
    $ref: ./paths/auth.yml#csrf
    tags:
      - Authentication
  /auth/session:
    $ref: ./paths/auth.yml#session
    tags:
      - Authentication
  /users:
    $ref: ./paths/users.yml
    tags:
      - Users
  /users/{userId}:
    $ref: ./paths/user.yml
    tags:
      - Users
  /verses/{verseId}:
    $ref: ./paths/verse.yml
    tags:
      - Verses
  /languages:
    $ref: ./paths/languages.yml
    tags:
      - Languages
  /languages/{code}:
    $ref: ./paths/language.yml
    tags:
      - Languages
  /languages/{code}/verses/{verseId}/translation:
    $ref: ./paths/translations.yml
    tags:
      - Translations
  /languages/{code}/verses/{verseId}/glosses:
    $ref: ./paths/verse-glosses.yml
```

```
    tags:
      - Glosses
  /languages/{code}/words/{wordId}:
    $ref: ./paths/language-words.yml
    tags:
      - Words
```

In the above example, I've added the `tags` field to each endpoint definition. Each endpoint is assigned one or more tags based on its category or functionality. You can define any custom tags that suit your API's structure and organization. In this case, I used tags like `Authentication`, `Users`, `Verses`, `Languages`, `Translations`, `Glosses`, and `Words`.

Feel free to modify the tags according to your specific requirements.